# ASP.NET Core Model Validation

*The Definitive Guide: Safeguarding Your Application's Data*

## 1. What is Model Validation?

Model validation is the process of confirming that the data submitted to your application meets a set of predefined rules *before* your application attempts to process it. In the ASP.NET Core pipeline, **validation occurs immediately after Model Binding**.

> **The Core Philosophy**
>
> Never trust user input. Validation acts as the primary gatekeeper, ensuring that malformed data, missing required fields, or out-of-bounds values are rejected with a helpful error message before they can cause exceptions in your business logic or corrupt your database.

## 2. The Two Main Approaches

In the .NET ecosystem, there are two primary ways to implement model validation: the built-in **DataAnnotations** and the popular third-party library **FluentValidation**.

### Approach A: DataAnnotations (The Built-in Way)

DataAnnotations use attributes directly on your model's properties. This is quick to set up and built directly into the framework.

```
public class UserRegistrationDto
{
    [Required(ErrorMessage = "Name is required.")]
    [StringLength(50, MinimumLength = 2)]
    public string Name { get; set; }

    [Required]
    [EmailAddress]
    public string Email { get; set; }

    [Range(18, 120, ErrorMessage = "You must be an adult to register.")]
    public int Age { get; set; }
}
```

### Approach B: FluentValidation (The Modern/Clean Way)

FluentValidation keeps validation logic completely separate from your models. This adheres to the Single Responsibility Principle and allows for much more complex, conditional validation rules.

```
// 1. The Model remains clean
public class UserRegistrationDto
{
    public string Name { get; set; }
    public string Email { get; set; }
    public int Age { get; set; }
}

// 2. The Validator contains the rules
public class UserRegistrationValidator : AbstractValidator<UserRegistrationDto>
{
    public UserRegistrationValidator()
    {
        RuleFor(x => x.Name).NotEmpty().Length(2, 50);
        RuleFor(x => x.Email).NotEmpty().EmailAddress();
        RuleFor(x => x.Age).InclusiveBetween(18, 120)
            .WithMessage("You must be an adult to register.");
    }
}
```

## 3. How it Works (The Execution Pipeline)

Validation does not happen in Middleware; it happens inside **Filters**.

1. **Model Binding Completes:** The raw JSON/Form is mapped to a C# object (e.g., `UserRegistrationDto`).
2. **Validation Runs:** The framework inspects the object. If using DataAnnotations, it reads the attributes. If using FluentValidation, it executes your validator classes.
3. **ModelState is Populated:** Any failed rules are added to a dictionary called `ModelState`, containing the property name and the error message.
4. **The Automatic Short-Circuit:** If your Controller is decorated with the `[ApiController]` attribute, a built-in filter called `ModelStateInvalidFilter` kicks in. If `ModelState.IsValid` is false, this filter instantly returns an HTTP 400 Bad Request with a standardized `ProblemDetails` JSON response, **preventing your controller action from ever executing.**

## 4. The "Things You Didn't Mention" (Advanced Concepts)

### A. Cross-Property Validation (The IValidatableObject Interface)

DataAnnotations are great for single properties, but what if a rule depends on two properties? For example, `EndDate` must be after `StartDate`. You can implement `IValidatableObject` directly on your model:

```
public class EventDto : IValidatableObject
{
    public DateTime StartDate { get; set; }
    public DateTime EndDate { get; set; }

    public IEnumerable<ValidationResult> Validate(ValidationContext context)
    {
        if (EndDate <= StartDate)
        {
            yield return new ValidationResult(
                "End Date must be greater than Start Date.",
                new[] { nameof(EndDate) } // Highlights the specific field
            );
        }
    }
}
```

> **Execution Order Warning**
>
> `IValidatableObject.Validate` only executes if **all** property-level DataAnnotations pass first. If a `[Required]` attribute fails, cross-property validation will not run.

### B. Custom Validation Attributes

If you find yourself writing the same complex regex or custom logic repeatedly, you can create your own reusable attribute by deriving from `ValidationAttribute`:

```
public class AllowedDomainAttribute : ValidationAttribute
{
    private readonly string _domain;
    public AllowedDomainAttribute(string domain) => _domain = domain;

    protected override ValidationResult IsValid(object value, ValidationContext context)
    {
        var email = value as string;
        if (email != null && email.EndsWith($"@{_domain}"))
            return ValidationResult.Success;

        return new ValidationResult($"Email must be from {_domain}.");
    }
}
```

### C. Server-Side vs. Client-Side Validation

If you are building an API, validation is strictly Server-Side. However, if you are using MVC Views or Razor Pages, ASP.NET Core can automatically generate HTML5 `data-val` attributes based on your DataAnnotations. When paired with jQuery Validation scripts, this provides **Client-Side Validation** (preventing the form submission without a page reload). *Always ensure server-side validation is enforced, as client-side validation can be bypassed by malicious users.*

## 5. Best Practices & Common Pitfalls

| Practice | Why it matters |
|---|---|
| **Don't mix business logic with validation** | Validation is for data shape and format (e.g., "Is this a valid email format?"). Checking if "Email already exists in the database" belongs in a Domain Service, not a validation attribute, to avoid unintended database hits during binding. |
| **Use ProblemDetails** | Ensure your API returns standard RFC 7807 `ProblemDetails` for validation errors. `[ApiController]` does this by default, ensuring a consistent error format for frontend developers. |
| **Prefer FluentValidation for APIs** | As APIs grow, DataAnnotations make models bloated. FluentValidation offers better unit testing, dependency injection (for advanced rules), and a cleaner architecture. |